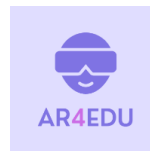




Co-funded by the
Erasmus+ Programme
of the European Union



Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра автоматизованих систем управління



Методичні вказівки
до лабораторної роботи №2
**«Створення досвіду доповненої реальності за допомогою можливостей WebXR Device
API та бібліотеки Three.js»**
з дисципліни
«Системи віртуальної та доповненої реальності»

Львів 2025

Методичні вказівки до лабораторної роботи №2 «Створення досвіду доповненої реальності за допомогою можливостей WebXR Device API та бібліотеки Three.js» з дисципліни «Системи віртуальної та доповненої реальності» для студентів спеціальності 122 «Комп'ютерні науки», освітньої програми «Комп'ютерні науки (Обчислювальний інтелект смарт-систем)» Укл. Глод С. І., Львів: Національний університет «Львівська Політехніка», 2025.

Методичні вказівки обговорено та схвалено на засіданні кафедри АСУ

Протокол № _____ від «___» _____ 2025р.

Завідувач кафедрою АСУ _____ *Теслюк В. М.*

Методичні вказівки обговорено та схвалено на засіданні методичної комісії базового напрямку підготовки

Протокол № _____ від «___» _____ 2025р.

Мета роботи: ознайомлення з концепцією доповненої реальності (AR) та її застосуванням у вебсередовищі; вивчення можливостей Three.js та WebXR Device API для створення інтерактивних AR-досвідів; набуття практичних навичок у реалізації 3D-сцен, створенні власних 3D-об'єктів, завантаженні та інтеграції моделей у форматі GLTF, а також розміщенні довільного 3D-контенту у реальному просторі

Лабораторне завдання: завдання другої лабораторної роботи полягає у створенні інтерактивного досвіду доповненої реальності (AR) у вебсередовищі з можливістю розміщення віртуальних об'єктів у реальному просторі; реалізації Web-XR-застосунків з можливістю перегляду створеного 3D-контенту, перегляду завантажених 3D-моделей, додавання 3D-контенту у реальний простір та тестування готових проєктів за допомогою мобільних пристроїв з операційною системою Android. Для виконання лабораторного завдання використовуються бібліотека Three.js (дозволяє швидко і легко реалізовувати власний тривимірний контент) та WebXR Device API (група стандартів для реалізації XR-досвідів). Досвід доповненої реальності повинен надавати користувачеві можливість переглядати створені 3D-об'єкти, переглядати завантажені GLTF-моделі та можливість розміщувати 3D-об'єкти і 3D-моделі у реальному просторі.

Теоретичні відомості

WebAR (Web-based Augmented Reality) – це технологія доповненої реальності, яка працює безпосередньо у браузері без необхідності встановлення додаткових застосунків. Використовуючи WebXR Device API, WebGL, Three.js та інші інструменти, розробники можуть створювати інтерактивні AR-застосунки, які працюють на різних пристроях, включаючи смартфони, планшети та VR/AR-гарнітури. WebAR поєднує можливості вебтехнологій та 3D-графіки, забезпечуючи швидке та зручне впровадження власного AR-контенту. Завдяки цьому користувачі можуть миттєво отримати доступ до бажаного досвіду доповненої реальності без встановлення додаткових програм – достатньо просто відкрити посилання у браузері та розпочати взаємодію.

Традиційно доповнена реальність реалізується через нативні застосунки для iOS (бібліотеки для реалізації AR: ARKit, SceneKit, RealityKit, Metal) чи Android (бібліотеки для реалізації AR: ARCore, Sceneform), однак WebAR має кілька ключових переваг:

- *Миттєве розгортання* – користувачам не потрібно завантажувати застосунки з App Store або Google Play, достатньо відкрити вебсторінку.
- *Кросплатформеність* – підтримка різних пристроїв та операційних систем.
- *Доступ до усіх веб-API* – можливість використання інших вебтехнологій, зокрема WebGL для рендерингу 3D-графіки, доступ до камери, геолокації тощо.
- *Універсальність коду* – єдина кодова база для різних платформ із мінімальними змінами для адаптації під AR- та VR-середовища.

Завдяки цим перевагам WebAR є ефективним підходом до створення доповненої реальності, який поєднує гнучкість вебтехнологій із можливостями 3D-графіки. Одним із

ключових інструментів для реалізації WebAR є **WebXR Device API**, який забезпечує взаємодію з пристроями доповненої та віртуальної реальності безпосередньо у браузері.

WebXR, в основі якого лежить WebXR Device API, забезпечує функціональність, необхідну для перенесення доповненої і віртуальної реальності в Інтернет ('X' у XR – це свого роду алгебраїчна змінна, яка може означати все, що завгодно в діапазоні імерсивних вражень – наприклад, mixed reality, extended reality, cross reality тощо). WebXR – це група стандартів, які використовуються разом для підтримки рендерингу 3D-сцен на апаратне забезпечення, призначене для представлення віртуальних світів (VR) або для додавання графічного контенту до реального світу (AR). WebXR Device API реалізує ядро набору функцій WebXR, керуючи вибором пристроїв виводу, рендерингом 3D-сцени на вибраний пристрій з відповідною частотою кадрів і управлінням векторами руху, створеними за допомогою контролерів. Пристрої, сумісні з WebXR, включають повністю імерсивні 3D-гарнітури з відстеженням руху та орієнтації; окуляри, які накладають графіку на реальну сцену, а також кишенькові мобільні пристрої, які доповнюють реальність, захоплюючи світ за допомогою камери та доповнюючи реальну сцену графічним контентом (наприклад, Google Daydream, HTC Vive, Magic Leap One, Microsoft HoloLens, Oculus Rift, Samsung Gear VR, гарнітури Windows Mixed Reality, імерсивні гарнітури, AR-окуляри, ARCore-сумісні пристрої, смартфони з підтримкою AR тощо).

Важливо звернути увагу на те, що WebXR не є технологією рендерингу і не надає функцій для управління 3D-даними або їх виведення на дисплей. Це важливий факт, про який слід пам'ятати. WebXR керує таймінгом, плануванням і різними точками зору, важливими для малювання (створення) сцени, особливо правильною частотою кадрів, проте WebXR не знає, як завантажувати і керувати моделями, як їх рендерити, текстурувати тощо. Ця частина повністю залежить від розробника. На щастя, WebGL та різноманітні фреймворки і бібліотеки на основі WebGL значно спрощують процес реалізації бажаного функціоналу. Отже, основні функції WebXR включають:

- *Розпізнавання сумісних VR/AR-пристроїв* – WebXR автоматично визначає доступні пристрої доповненої чи віртуальної реальності. Це дозволяє створювати універсальні рішення, які працюють на різних апаратних платформах.
- *Керування відтворенням 3D-сцени* – стосується часу (коли саме повинен відбуватися рендеринг сцени), швидкості (наскільки швидко зарендериться сцена) і точок зору для відтворення сцени на дисплеї пристрою.
- *Доступ до сенсорів пристрою* – WebXR отримує дані з акселерометрів, гіроскопів та інших датчиків, що дозволяє визначати положення, орієнтацію та рух користувача у просторі (рух мобільного пристрою). Це забезпечує

реалістичне відображення AR-об'єктів відповідно до зміни кута огляду або положення пристрою.

- *Підтримка пристроїв введення* – WebXR обробляє сигнали з різних пристроїв керування, таких як VR-контролери, геймпади змішаної реальності, рукавички з тактильним зворотним зв'язком та навіть жести користувача. Це дає змогу розробникам створювати інтерактивні сценарії взаємодії у віртуальному або змішаному середовищі.
- *«Environmental Understanding» (розуміння навколишнього середовища)* – WebXR може аналізувати фізичний простір навколо користувача, використовуючи камери та інші датчики. Це включає в себе розпізнавання поверхонь, визначення джерел світла та перешкод у просторі. Завдяки цьому в AR-режимі наші мобільні пристрої «розуміють», яких змін зазнають AR-об'єкти, коли вони віддаляються, зближуються або збільшуються у власних розмірах. Це дозволяє створити більш природну взаємодію з віртуальними об'єктами.

Іншою важливою складовою WebXR є базові поняття XR. Вони допомагають зрозуміти деякі базові концепції, які визначають роботу з віртуальною і доповненою реальністю.

Поле зору (Field of View, FOV)

Поле зору визначає, наскільки широкий кут огляду має користувач у віртуальному або доповненому (камера мобільного пристрою або AR-окуляри) середовищі. У людини загальний кут огляду становить приблизно 200° - 220° , з яких 135° припадає на одне око, а близько 115° – це область бінокулярного зору, який забезпечує глибину сприйняття (рис. 1).

Для досягнення реалістичного ефекту занурення XR-гарнітури мають відповідати цим параметрам. Базові гарнітури забезпечують FOV від 90° , тоді як найкращі моделі досягають 150° . Розширення поля зору підвищує рівень занурення, але також збільшує вагу і вартість пристрою (*Meta Quest 2* - 97° , *Meta Quest 3* – 110°).

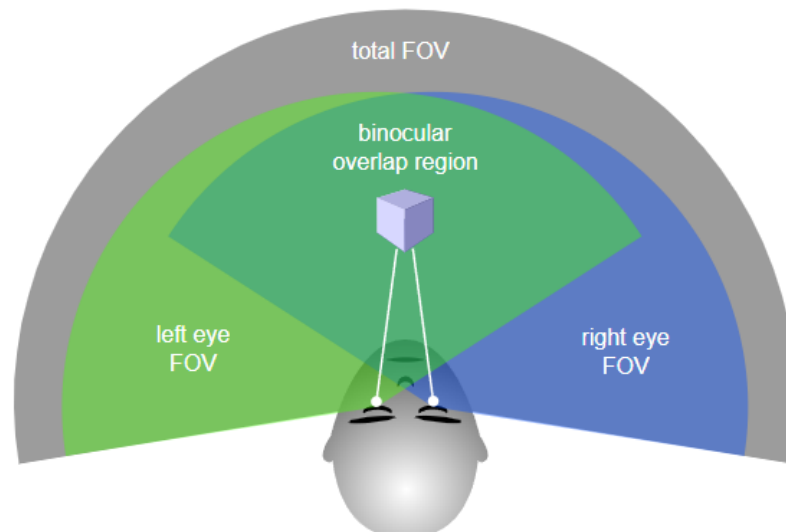


Рис. 1. Концепція Field of View

Ступені свободи (*Degrees of Freedom, DoF*)

Ступені свободи визначають, як користувач може переміщуватися та взаємодіяти у віртуальному просторі.

Обертальні рухи (*Rotational DoF, 3DoF*) дозволяють користувачеві змінювати положення голови навколо трьох осей: нахилити вперед-назад (*Pitch*), повертати ліворуч-праворуч (*Yaw*) та нахилити вліво-вправо (*Roll*). Гарнітури, що підтримують 3DoF, обмежуються лише зміною напрямку погляду без можливості переміщення у просторі (даний принцип підтримується в мобільних AR-застосунках – користувач може обертати пристрій і змінювати кут огляду, але не може переміщуватись у просторі, щоб змінити перспективу).

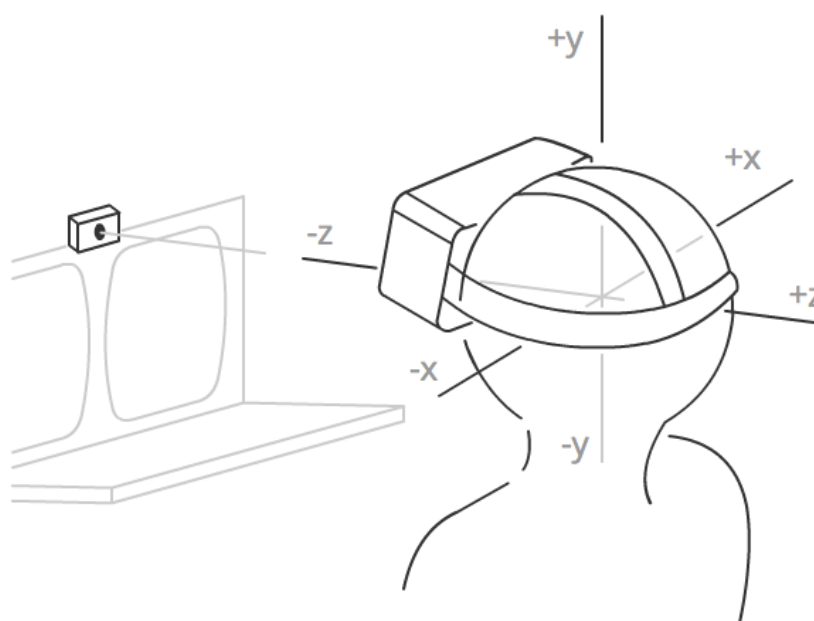


Рис. 2. Система координат, що використовується у XR для визначення положення і руху користувача у просторі (концепція 6DoF)

Лінійні переміщення (*Translational DoF, 6DoF*) (рис. 2) додають можливість рухатися у просторі в трьох напрямках: вперед-назад, вліво-вправо та вгору-вниз. Це дозволяє користувачеві не тільки рухати головою, а й фізично переміщуватися у віртуальному середовищі, що суттєво покращення рівень імерсивності (6DoF у AR підтримується лише спеціальними AR-гарнітурами (HoloLens, Magic Leap) та смартфонами з підтримкою ARKit або ARCore, які можуть відстежувати не тільки повороти, а й фізичне переміщення користувача).

WebXR пропонує розробникам підтримку сеансів доповненої і віртуальної реальності, використовуючи той самий API. Тип сеансу вказується під час його реалізації.

WebXR підтримує два режими VR-сесій: *inline* та *immersive*. У режимі *inline* віртуальна сцена відображається безпосередньо у браузері, без необхідності використання спеціального XR-обладнання. Натомість режим *immersive-vr* вимагає XR-гарнітуру та замінює весь

навколишній світ користувача цифровим середовищем, відображаючи сцену через дисплеї гарнітури для кожного ока.

Доповнена реальність накладає віртуальні об'єкти на реальний світ, створюючи змішане середовище, де цифровий контент інтегрується з фізичним простором. Оскільки AR завжди є повністю імерсивним досвідом, WebXR використовує для нього єдиний режим *immersive-ar*.

Незалежно від пристрою, WebXR підтримує як VR-, так і AR-сесії, а рендеринг сцени у доповненій реальності майже не відрізняється від VR. Основна різниця полягає в тому, що у VR сцена повністю замінює реальний світ, тоді як у AR фон залишається незмінним (реальний світ).

Хоча WebXR є ключовим API для роботи з VR/AR-середовищем, важливо розуміти, що він не є технологією рендерингу. Його основне завдання – забезпечувати взаємодію між браузером та XR-пристроями, керувати сесіями та відстежувати положення користувача у просторі. WebXR не відповідає за завантаження, управління, текстурування чи анімацію 3D-моделей. Усі візуальні ефекти, включаючи рендеринг об'єктів, текстуризацію та освітлення, обробляються іншими технологіями, такими як WebGL, Three.js, Babylon.js та іншими. Також WebXR Device API має деякі обмеження, а саме обмежену доступність (підтримується лише через Microsoft Edge та Google Chrome) і працює лише у захищеному середовищі (HTTPS), що гарантує безпечну взаємодію з XR-пристроями. Таким чином, при розробці WebXR-застосунків слід використовувати додаткові інструменти для створення тривимірної сцени, а WebXR буде лише засобом її інтеграції у віртуальну або доповнену реальність.

Оскільки рендеринг 3D-графіки, а особливо змішаної реальності, включає складні математичні обчислення, управління даними та інші технічні завдання, безпосереднє використання WebGL для створення сцени зазвичай не є оптимальним рішенням. Натомість більшість розробників використовують спеціалізовані фреймворки та бібліотеки, побудовані поверх WebGL, що значно спрощує процес розробки.

Однією з ключових переваг використання таких фреймворків є інтеграція віртуальної камери. Оскільки OpenGL і WebGL не надають вбудованих інструментів для перегляду сцени з камери, використання бібліотек, які імітують цю функціональність, значно спрощує реалізацію навігації у віртуальному просторі.

WebGL є основною технологією рендерингу для WebXR, тому перед початком роботи з доповненою чи віртуальною реальністю варто ознайомитися з відповідними фреймворками та бібліотеками, щоб ефективно використовувати їх можливості.

Three.js

Three.js – це популярна 3D-бібліотека, яка спрощує роботу з WebGL та намагається максимально спростити реалізацію тривимірного контенту на вебсторінці. Вона широко використовується у веброзробці для створення інтерактивної тривимірної графіки, включаючи застосунок з підтримкою віртуальною та доповненої реальності, а також допомагає уникнути написання низькорівневого коду, який необхідний для WebGL. WebGL – це низькорівнева система, яка здатна малювати лише точки, лінії та трикутники. Щоб створити щось корисне за допомогою WebGL, зазвичай потрібно досить багато коду, і саме тут вступає в дію бібліотека – Three.js. Вона обробляє такі речі, як сцени, світло, тіні, матеріали, текстури, тривимірну математику – все те, що розробникам довелося б писати самостійно, якби вони використовували WebGL напряду.

Основною перевагою бібліотеки Three.js є її простота у використанні та широкий набір функцій. Бібліотека надає зручні інструменти для створення геометричних фігур, текстурування, додавання джерел освітлення, створення матеріалів та реалізації фізичних ефектів. Крім того, Three.js підтримує імпорт популярних 3D-форматів, таких як **GLTF**, **OBJ**, **FBX** та інші, що дозволяє легко інтегрувати моделі, створені у спеціалізованих 3D-редакторах.

Three.js також надає вбудовану підтримку для роботи з камерами, що особливо важливо при створенні WebXR-застосунків. Це дозволяє розробникам зручно реалізовувати огляд сцени з певної точки або створювати навігацію у віртуальному середовищі. Для анімації можна використовувати як вбудовані механізми анімації, так і спеціалізовані бібліотеки, що працюють у зв'язці з Three.js.

У контексті WebXR, Three.js відіграє важливу роль, оскільки дозволяє створювати сцени з урахуванням параметрів XR-пристроїв, таких як поле зору (FOV), глибина сцени та підтримка об'ємних ефектів. Завдяки цьому Three.js є чудовим вибором для створення застосунків у сферах віртуальної та доповненої реальності.

Варто розглянути загальну структуру застосунка на Three.js. Розробка з використанням Three.js передбачає створення групи об'єктів і їх взаємозв'язок у межах сцени. Кожен елемент 3D-середовища має своє місце у цій структурі.

Щоб краще зрозуміти принципи побудови сцени, можна скористатися схематичним представленням типової структури Three.js-застосунка, що включає основні компоненти та їх взаємодію (рис. 3).

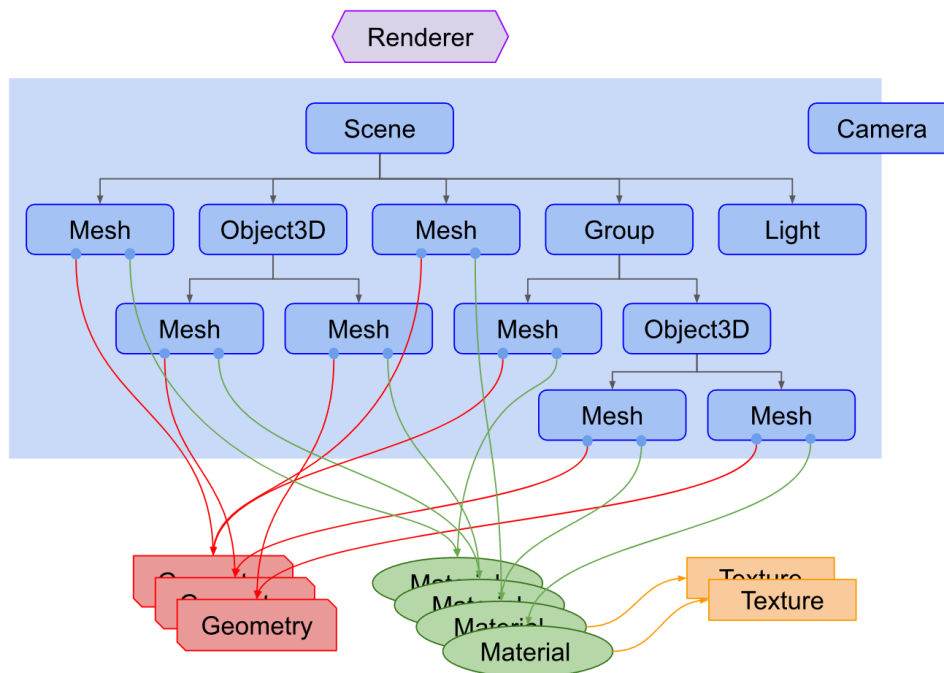


Рис. 3. Типова структура звичайного Three.js-застосунку

У Three.js будь-який 3D-застосунок будується навколо рендерера (Renderer), який відповідає за відображення сцени. Він отримує сцену (Scene) та камеру (Camera) і рендерить ту частину простору, що потрапляє у поле зору камери, у вигляді 2D-зображення на полотні (canvas).

Структура сцени базується на графі сцени (scenegraph) – ієрархічному дереві об'єктів, що включає сітчасті об'єкти (Mesh – полігональна структура), світлові джерела (Light), групи об'єктів (Group) та інші елементи. Об'єкти у сцені розташовуються відносно своїх батьківських елементів, що дозволяє легко керувати взаємопов'язаними структурами.

Камера у Three.js може бути частиною графа сцени або існувати окремо, при цьому вона може бути дочірнім об'єктом іншого елемента, змінюючи своє положення відповідно до нього. Джерела світла (Light) освітлюють сцену, створюючи реалістичне відображення об'єктів.

Сітчасті об'єкти або меші (Mesh) визначають геометрію об'єкта та його зовнішній вигляд. Вони поєднують геометрію (Geometry), що містить дані про вершини (наприклад, сфери, куби або довільні форми), та матеріали (Material), які визначають колір, текстуру та властивості поверхні.

Таким чином, структура Three.js-застосунку складається з взаємопов'язаних об'єктів, що формують сцену та забезпечують її рендеринг у браузері.

Окрім Three.js, активно використовуються й інші бібліотеки для роботи з WebXR. Babylon.js – це фреймворк для створення високоякісної 3D-графіки у браузері, який пропонує розширені можливості для роботи з фізикою, анімацією та тінями, а також має вбудовану підтримку WebXR. Він є гарним вибором для розробників, які прагнуть створювати складні

та реалістичні сцени. MindAR.js – спеціалізована бібліотека для роботи з доповненою реальністю (AR), орієнтована на швидке та ефективне розпізнавання зображень і маркерів у вебзастосунках. Вона дозволяє легко створювати інтерактивні AR-сценарії, що працюють безпосередньо у браузері без потреби у складному налаштуванні. Використання цих бібліотек дозволяє розробникам вибирати інструменти, що найкраще відповідають їхнім завданням – від створення реалістичних VR-сцен до легких AR-застосунків.

Життєвий цикл застосунку WebXR

Робота WebXR-застосунку складається з кількох ключових етапів (рис. 4), що визначають процес ініціалізації, запуску та завершення AR-сесії (у нашому випадку).

1. Перевірка підтримки WebXR – на першому етапі застосунк переверіє, чи пристрій підтримує WebXR. Якщо підтримка доступна, можна перейти до наступного кроку.
2. Відображення кнопки запуск AR – у разі успішного виявлення AR-пристрою інтерфейс показує кнопку, наприклад, «Start AR», яка дозволяє користувачеві розпочати AR-сесію.
3. Запит AR-сесії – після натискання кнопки користувачем застосунк надсилає запит на запуск *immersive-ar* сесії. Цей запит повинен бути ініційований у відповідь на дію користувача, оскільки браузери зазвичай блокують автоматичний запуск XR-контенту.

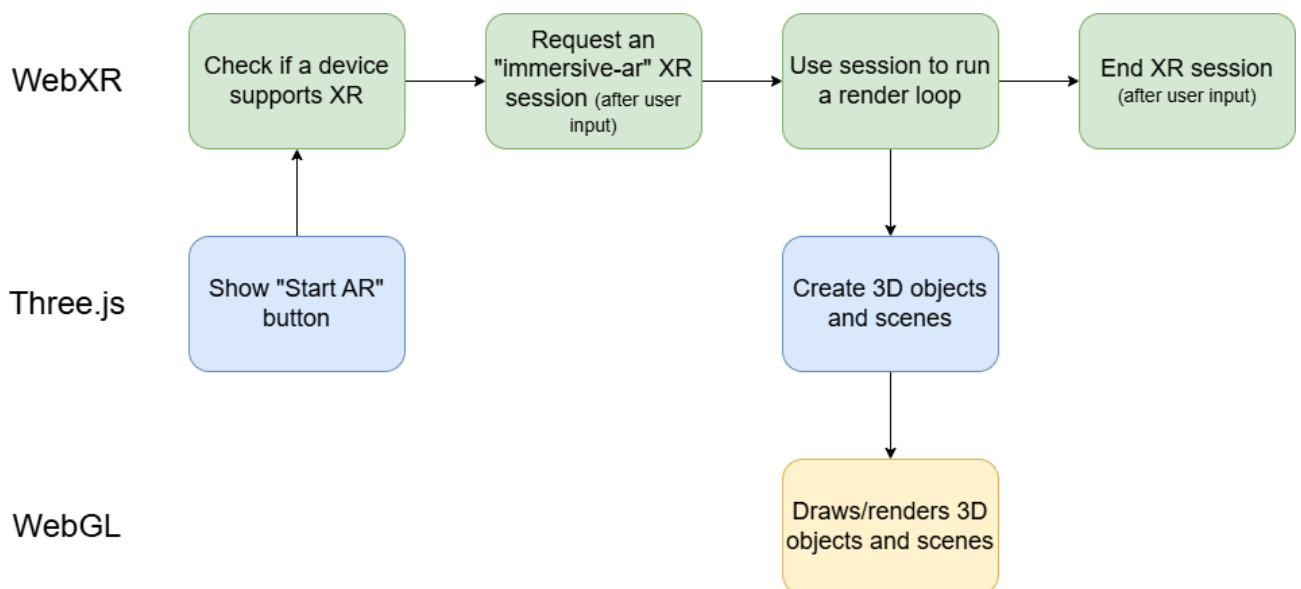


Рис. 4. Життєвий цикл WebXR-застосунку

4. Запуск циклу рендерингу – якщо запит успішно оброблено, WebXR створює AR-сесію і запускає рендеринг-цикл, у якому сцена оновлюється в реальному часі відповідно до рухів користувача та взаємодій з об’єктами.
5. Створення 3D-об’єктів і сцени – під час циклу бібліотека Three.js генерує 3D-об’єкти та інтерактивні елементи сцени, що будуть відображатися у AR-середовищі.

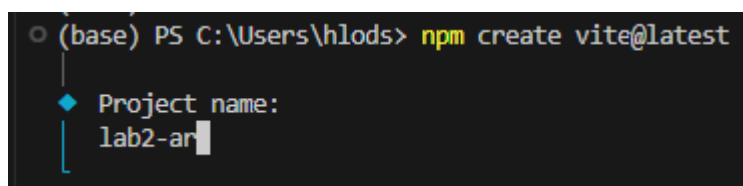
6. Рендеринг сцени/об'єктів – за допомогою WebGL, бібліотека Three.js відображає створені об'єкти, текстури та освітлення, формуючи фінальний простір AR-середовища.
7. Завершення сесії – користувач може завершити AR-сесію у будь-який момент, після чого рендеринг припиняється, і застосунок повертається до початкового стану.

Такий життєвий цикл є типовим для WebXR-застосунків. Він визначає загальну послідовність роботи застосунків та їхню інтеграцію з додатковими бібліотеками і WebGL для створення повноцінного XR-досвіду (AR/VR).

Хід роботи

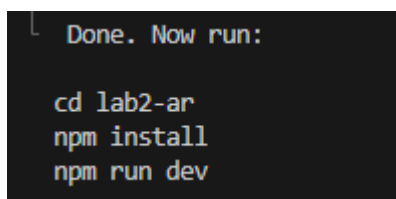
1. Перш ніж розпочати виконувати завдання другої лабораторної роботи зупинимось на деяких ключових моментах. По-перше, перед початком роботи встановіть *Node.js* (версію 18+ або 20+), який необхідний для керування залежностями та запуску нашого інструменту збірки. По-друге, приклади виконання завдань другої лабораторної роботи були розгорнуті у Visual Studio Code через використання інструменту збірки [Vite](#) (ви можете використовувати довільне середовище розробки і підхід до збірки ваших проєктів).
2. Розпочнемо зі створення нового проєкту через Vite. Ви можете самостійно ознайомитись [з покроковою інструкцією на офіційному сайті](#) або ж слідувати наступним крокам. У VS Code відкрийте новий термінал та виконайте команду (рис. 5):
`npm create vite@latest`

Після виконання команди ви запустите свого роду скрипт з доволі зрозумілими вказівками відносно того, як створити власний проєкт за допомогою інструменту Vite. Вам необхідно дати назву вашому проєкту, обрати фреймворк (*обираємо Vanilla*) та обрати JavaScript. Після цього, ви отримаєте нові вказівки (рис. 6).



```
(base) PS C:\Users\hlods> npm create vite@latest
└─ Project name:
   lab2-ar
```

Рис. 5. Результат виконання команди `npm create vite@latest`



```
Done. Now run:

cd lab2-ar
npm install
npm run dev
```

Рис. 6. Наступні вказівки щодо налаштування вашого проєкту

3. Виконайте наступні команди з отриманих вказівок. Команда `npm install` – встановлює усі необхідні пакети і бібліотеки та готує середовище до запуску через команду – `npm run dev` (ви також можете використовувати команду `npx vite --host`). Якщо все було налаштовано коректно, тоді ви успішно зможете запустити ваш проєкт (рис. 7) та перевірити його поточний стан, використавши будь-яке з доступних посилань (рис. 8). Також ви можете дослідити вміст вашого проєкту, відкривши теку із заданою назвою у середовищі VS Code (рис. 9).

```
VITE v6.2.2 ready in 221 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Рис. 7. Проєкт було успішно запущено

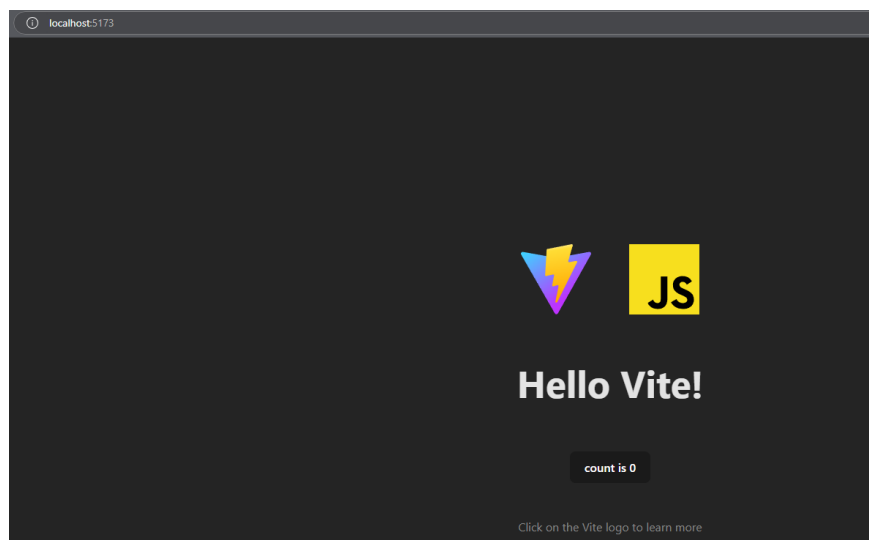


Рис. 8. Базова сторінка розгорнутого проєкту через інструмент збірки Vite

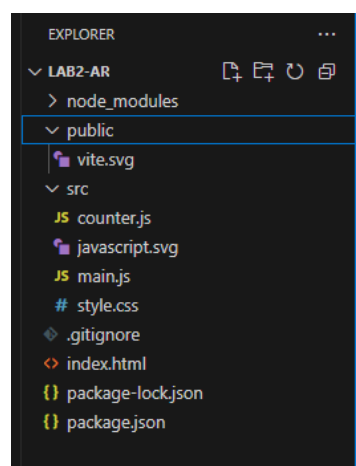


Рис. 9. Базова структура Vite-проєкту

4. Якщо ви уважно прочитали теоретичні відомості, тоді ви вже знаєте, що WebXR, який потрібен нам для розгортання AR-застосунку, працює лише у захищеному середовищі – HTTPS. Тому, вам слід внести деякі зміни у ваш проєкт. Використовуючи дане посилання: [@vitejs/plugin-basic-ssl - npm](https://www.npmjs.com/package/@vitejs/plugin-basic-ssl), відшукайте необхідну прм-команду, а тоді у теці з вашим проєктом виконайте її (рис. 10). Дана команда встановить у ваш проєкт певний плагін, а саме `@vitejs/plugin-basic-ssl`, який дасть вам змогу використовувати захищене середовище для ваших проєктів.

```
(base) PS C:\Users\hlods\lab2-ar> npm i @vitejs/plugin-basic-ssl

added 1 package, and audited 12 packages in 2s

3 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Рис. 10. Результат успішного виконання команди по встановленню плагіна

5. Далі створіть новий файл у вашому проєкті – `vite.config.js`, скопіюйте необхідний код з даного репозиторію: [осць](#), збережіть файл (рис. 11) та спробуйте запустити ваш проєкт. Якщо усе виконано коректно, тоді у вас буде схожий результат (рис. 12).

```
# style.css vite.config.js x
vite.config.js > [vite] default
1 // vite.config.js
2 import basicSsl from '@vitejs/plugin-basic-ssl'
3
4 export default {
5   plugins: [
6     basicSsl({
7       /** name of certification */
8       name: 'test',
9       /** custom trust domains */
10      domains: ['*.custom.com'],
11      /** custom certification directory */
12      certDir: '/Users/.../.devServer/cert',
13    }),
14  ],
15 }
```

Рис. 11. Вміст створеного файлу `vite.config.js`

```
(base) PS C:\Users\hlods\lab2-ar> npx vite --host
20:32:52 [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v6.2.2 ready in 551 ms

→ Local: https://localhost:5173/
→ Local: https://vite.custom.com:5173/
→ Network: https://192.168.1.84:5173/
→ press h + enter to show help
```

Рис. 12. Запуск Vite-проєкту із захищеним HTTPS-з'єднанням

6. Перш ніж перейти до виконання лабораторних завдань виконаємо ще кілька налаштувань. Відкрийте ваш файл `style.css` та замініть стандартний вміст даного файлу на наступний:

```
body {
  margin: 0;
  background-color: #000;
  color: #fff;
  font-family: Monospace;
  font-size: 16px;
  line-height: 24px;
  overscroll-behavior: none;
}
a {
  color: #ff0;
  text-decoration: none;
}
a:hover {
  text-decoration: underline;
}

button {
  cursor: pointer;
  text-transform: uppercase;
}
canvas {
  display: block;
}
a, button, input, select {
  pointer-events: auto;
}
```

7. Тепер видаліть файл `counter.js` та залиште у файлі `main.js` лише перший рядок коду, що стосується файлу `style.css`. Також зверніть увагу на фінальний вміст файлу `index.html` (рис. 13). *Примітка:* зверніть увагу, що завдань буде декілька, тому напевно доцільніше буде використовувати декілька `.js`-файлів, ніж кожного разу вводити `main.js`.



```
<> index.html > ...
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7      <title>Lab 2 AR</title>
8    </head>
9    <body>
10     <div id="app"></div>
11     <script type="module" src="/src/main.js"></script>
12   </body>
13 </html>
14
```

Рис. 13. Необхідний вміст файлу `index.html`

8. Завершальним етапом налаштування вашого проєкту буде встановлення пакету `Three.js`, який є основним у даній лабораторній роботі. У терміналі, перейдіть до теки з

вашим проєктом та виконайте команду – `npm install three`. У разі успішного результату виконання команди – процес налаштування проєкту є завершеним.

9. **Перше завдання** другої лабораторної роботи полягає у створенні *простого досвіду доповненої реальності* за допомогою Three.js та WebXR API. Ваш застосунок повинен активувати WebXR, ініціалізувати 3D-сцену, камеру, об'єкт рендерингу та світло, а також містити інтерактивну кнопку для входу в AR-режим. Ваша сцена повинна містити три 3D-об'єкти: дані об'єкти ви повинні створити згідно **вашого № індивідуального завдання (варіанту)**. Об'єкти повинні мати власні матеріали, кольори та базову анімацію. Не забувайте про розміщення об'єктів. Вони повинні розміщуватись на певній відстані від користувача, тобто від камери мобільного пристрою.

Приклад виконання першого завдання: <https://gist.github.com/labs-ar-vr/9b191402b2007d2e559acd9a34a0ebe6> (тут ви знайдете приклад програмного коду з трьома різними фігурами з базовою анімацією та різними матеріалами).

Для виконання першого завдання корисними будуть наступні посилання на документацію Three.js:

- <https://threejs.org/docs/index.html#api/en/> – це посилання на загальну документацію Three.js; у лівій частині вікна ви знайдете загальну ієрархію по документації (важливі пункти для виконання завдання: *Geometries, Lights, Materials, Animation*)
- <https://threejs.org/manual/#en/materials> – це загальна стаття про матеріали у Three.js.
- <https://threejs.org/manual/#en/lights> – це загальна стаття по освітленню у Three.js. На вебсторінці є вбудовані редактори освітлення, які допоможуть вам визначитись з вибором того чи іншого об'єкта освітлення для ваших застосунків.
- https://www.youtube.com/watch?v=q0jgqeS_ACM&ab_channel=Udacity – це навчальне відео краще дасть зрозуміти вам як працюють кути Ейлера в Three.js та як побудувати базову анімацію.

10. Для тестування вашого застосунку виконайте команду: `npx vite --host` та оберіть *Network-посилання*. Є два основних підходи до тестування ваших застосунків:

- a. На вебсторінці, через інструменти розробника, використовуючи розширення WebXR API Emulator (рис. 14) (*через певні проблеми з оновленнями розширення/політикою Google дане розширення недоступне у Google Chrome на даний момент, проте ви можете скористатись даним посиланням: [ось](#) та встановити дане розширення для Microsoft Edge*).

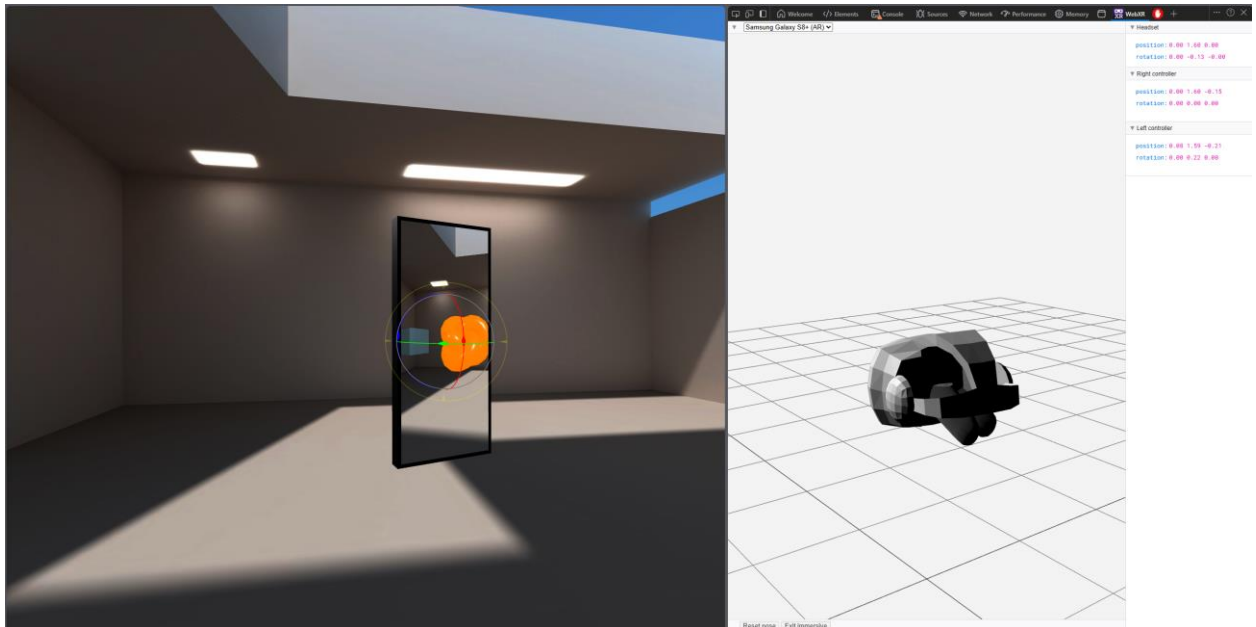


Рис. 14. Приклад тестування першого лабораторного завдання за допомогою розширення WebXR API Emulator

- б.** За допомогою вашого мобільного пристрою, зайдіть у Google Chrome (лише для пристроїв з операційною системою Android) та перейдіть до вкладки – **chrome://flags**, далі відшукайте усі можливі результати за ключовим словом **webxr**, знайдіть у списку WebXR Incubations та активуйте її (рис. 15). Після чого відкрийте ваше *Network-посилання* у браузері та натисніть на клавішу **Start AR**. Якщо все було налаштовано і реалізовано коректно, тоді у вас запуститься AR-режим з вашими 3D-об'єктами (рис. 16-17).

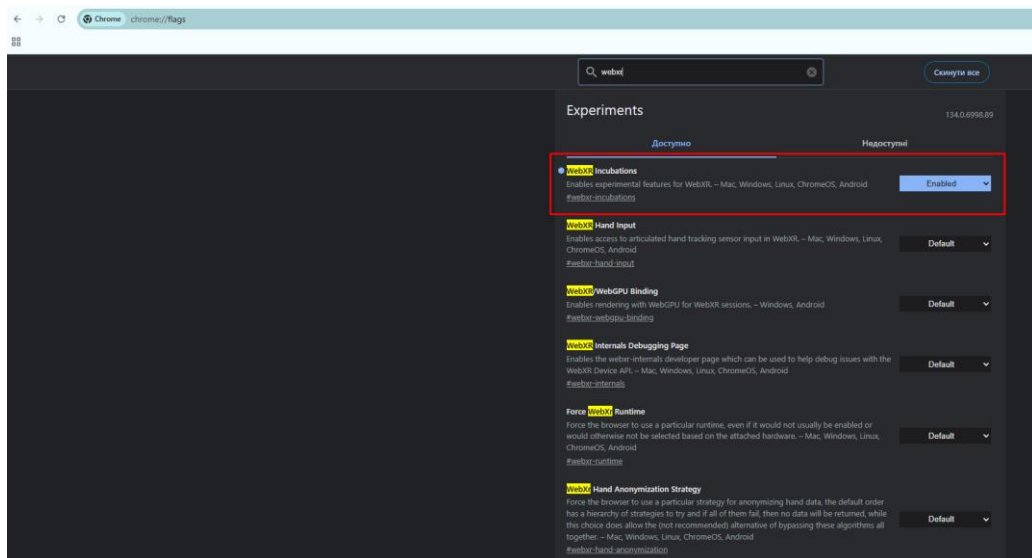


Рис. 15. Налаштування chrome flags для успішного запуску AR-режиму через WebXR

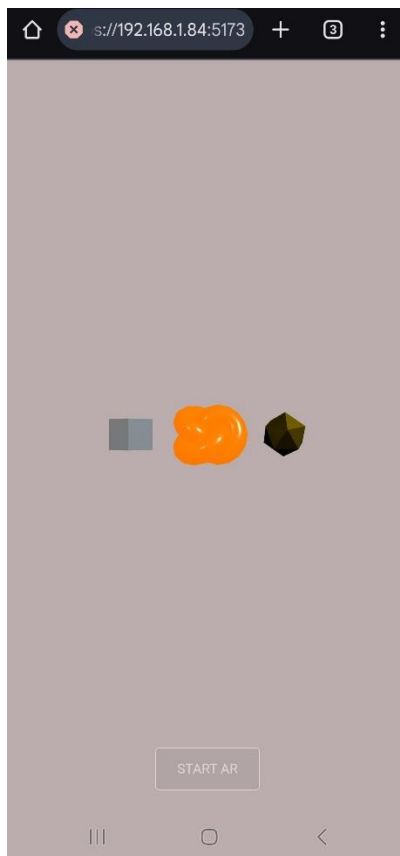


Рис. 16. Відкрита вебсторінка застосунку у мобільній версії Google Chrome

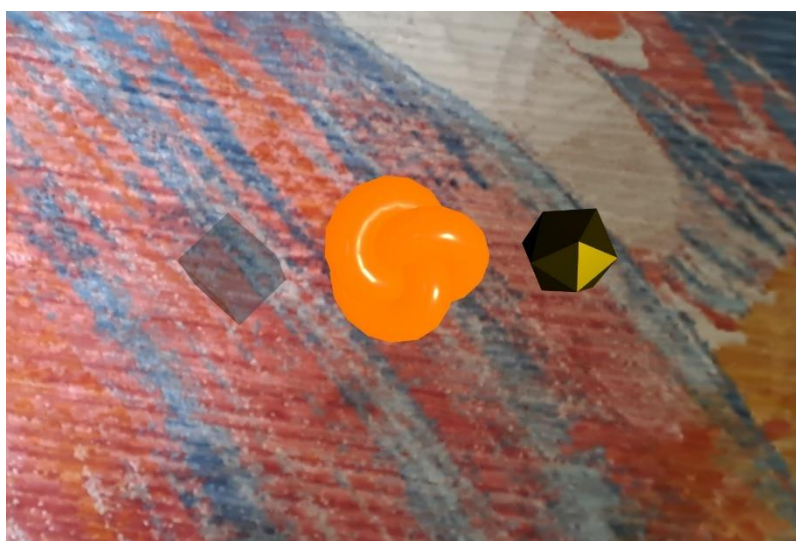


Рис. 17. Тестування першого завдання у AR-режимі зі створеними об'єктами

- 11. Друге завдання** поточної лабораторної роботи полягає у створенні досвіду доповненої реальності з додаванням на сцену GLTF 3D-моделі з реалізацією її анімації у реальному часі. Вам необхідно використати Three.js, WebXR та GLTFLoader для завантаження та відображення моделі у середовищі доповненої реальності. Ваш застосунок повинен активувати WebXR, ініціалізувати 3D-сцену, камеру, об'єкт рендерингу та світло, а також містити інтерактивну кнопку для входу в AR-режим. Далі

ви повинні завантажити довільну модель (згідно вашого № індивідуального завдання) із розширенням .gltf на вашу 3D-сцену.

Для того, щоб завантажити модель на сцену вам необхідно:

- Захостити ваші .gltf та .bin файли моделі у хмарному сховищі, наприклад AWS S3 (рис. 18). Безліч доступних моделей ви знайдете на знайомій для вас платформі Sketchfab. Проте, будьте пильними, не використовуйте багаторозмірні моделі.

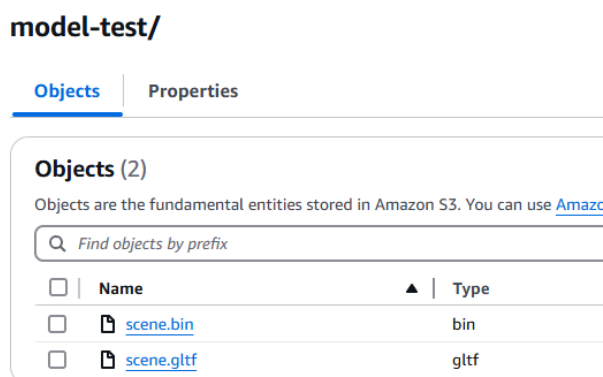


Рис. 18. Розміщені файли моделі у хмарному сховищі S3

- Далі використайте посилання на .gltf файл у вашому коді (пам'ятайте, що ваші файли .gltf та .bin мають бути публічними, для цього скористайтесь ACL, видозмініть Bucket Policy або скористайтесь AWS CLI). Ви можете працювати з будь-яким хмарним сховищем, головне ваше завдання – це успішно завантажити модель.
- Після процесу завантаження вам слід дослідити чи ваша модель коректно відображується, можливо вам доведеться змінити освітлення або створити необхідний матеріал та за допомогою функції – `model.traverse()` застосувати потрібні зміни для кращого відображення моделі на сцені.
- Після цього застосуйте просту анімацію для вашої моделі. Стаття про *MathUtils* може бути корисною для вас, якщо вам доведеться конвертувати градуси у радіани (<https://threejs.org/docs/#api/en/math/MathUtils>).

Приклад виконання другого завдання: <https://gist.github.com/labs-ar-vr/769c9ea9c9ecc54f27503f84d6fd32bf> (тут ви знайдете приклад програмного коду для завантаження GLTF-моделі на 3D-сцену Three.js; також описано приклад, де саме потрібно ініціалізовувати матеріали та використовувати функцію `traverse`).

Для виконання другого завдання корисними будуть наступні посилання на документацію Three.js:

- <https://threejs.org/manual/?q=GLt#en/load-gltf> – це посилання на статтю з прикладом як завантажити GLTF-модель та застосувати анімації до безлічі об'єктів.
- <https://threejs.org/docs/?q=G#examples/en/loaders/GLTFLoader> – це посилання на статтю про об'єкт GLTFLoader.

12. Підхід до тестування такий самий, як у попередньому завданні. Можете переглянути на вебсторінці чи все коректно завантажується (рис. 19-20), а тоді протестувати за допомогою мобільного пристрою (рис. 21).

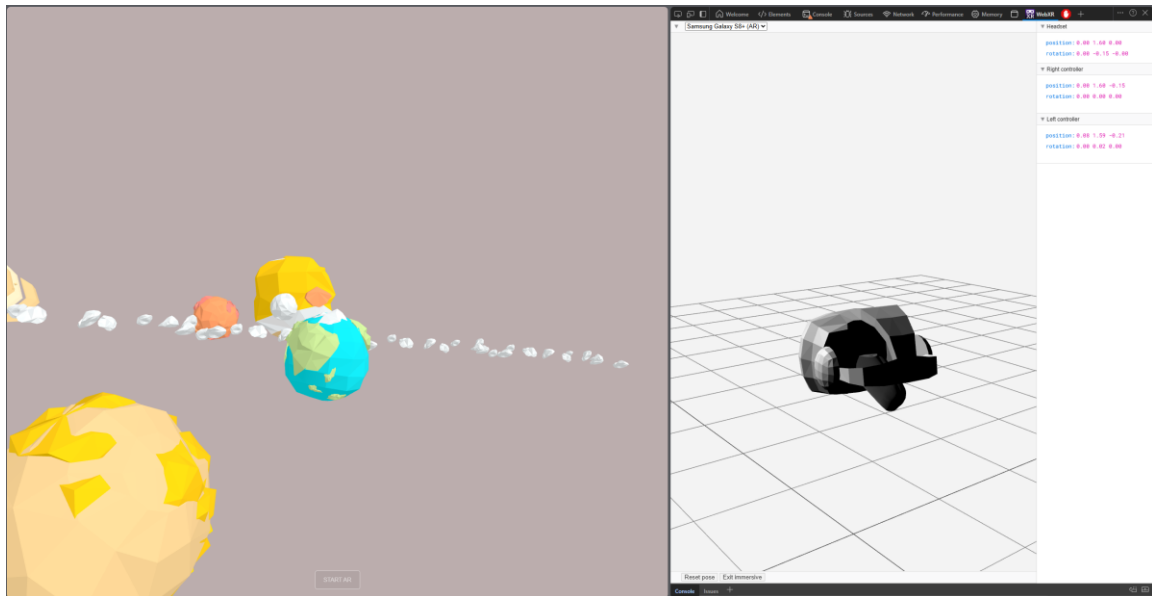


Рис. 19. Перегляд завантаженої моделі на вебсторінці

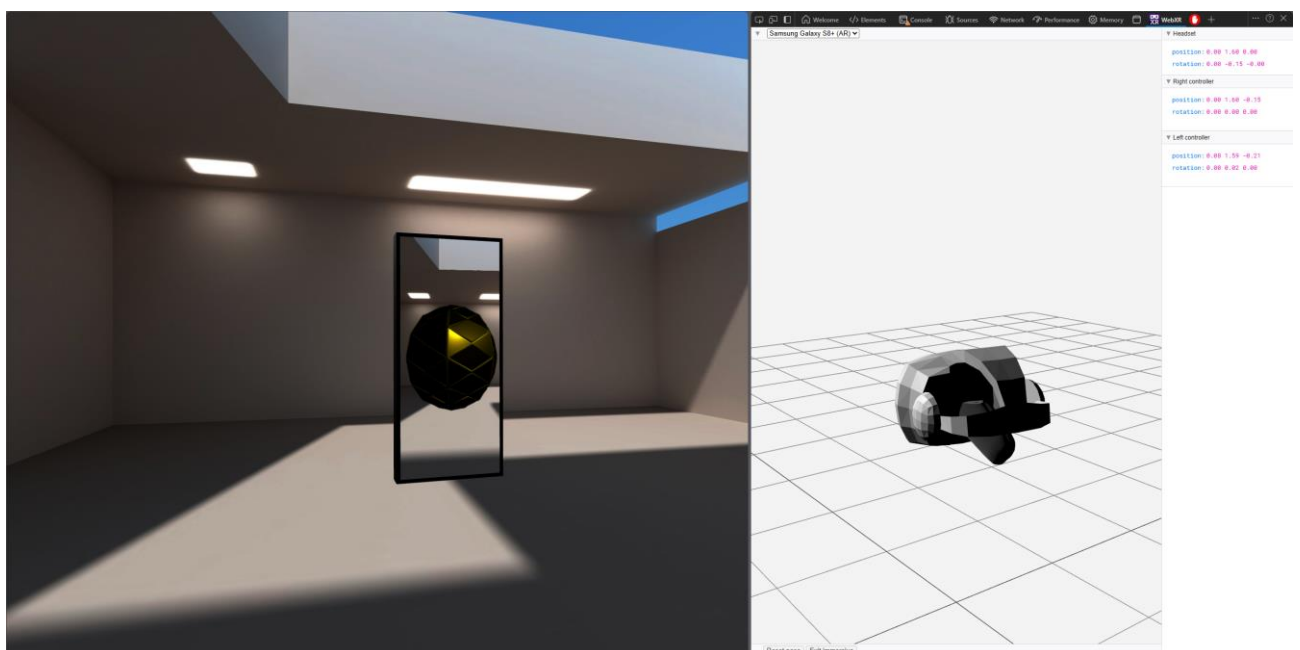


Рис. 20. Тестування AR-режиму з обраною моделлю за допомогою WebXR API Emulator



Рис. 21. Тестування другого завдання у AR-режимі за допомогою мобільного пристрою

13. Третє завдання поточної лабораторної роботи полягає у реалізації можливості розміщення 3D-об'єктів у реальному просторі за допомогою WebXR Hit Test. **Hit Testing** дозволяє визначати поверхні у реальному світі, а реалізований вами функціонал надаватиме можливість додавати об'єкти на ці поверхні при взаємодії з екраном мобільного пристрою. Застосунок повинен активувати WebXR, ініціалізувати 3D-сцену, камеру, об'єкт рендерингу та світло, а також містити інтерактивну кнопку для входу в AR-режим. Ви повинні створити мітку поверхні (reticle), яка слугуватиме візуальним індикатором для розміщення ваших об'єктів. Вам необхідно знайти поверхню у реальному світі та отримати необхідну інформацію (`requestHitTestSource`, `hitTestResults`). Після ініціалізації AR-режиму ви повинні мати змогу відносно мітки поверхні розмістити 3D-об'єкт (**об'єкт створюєте відповідно до № вашого індивідуального завдання**).

Приклад виконання третього завдання: <https://gist.github.com/labs-ar-vr/7f6b018dfd89f122c706aef9565022cb> (тут ви знайдете приклад програмного коду для реалізації Hit Test у AR-режимі з можливістю додавання на сцену розроблених 3D-об'єктів).

14. Четверте завдання полягає у реалізації можливості розміщення довільної моделі у реальному просторі за допомогою WebXR Hit Test. Завдання дуже схожі, проте у цьому

завдання замість 3D-об'єкту, вам необхідно завантажити та розмістити модель. Зверніть увагу на розміри моделі, освітлення та матеріали, щоб представлення моделі було настільки ж якісним, як у другому завданні.

Приклад виконання четвертого завдання: <https://gist.github.com/labs-ar-vr/18995d556c08be05c25b61f274c82828> (тут ви знайдете приклад програмного коду для реалізації Hit Test у AR-режимі з можливістю додавання на сцену довільної GLTF-моделі).

Для виконання третього та четвертого завдань корисними будуть наступні посилання:

- <https://github.com/immersive-web/hit-test/blob/master/hit-testing-explainer.md> – це стаття, яка пояснює як працює Hit Testing у WebXR Device API.
- <https://web.dev/articles/ar-hit-test> – це стаття, яка описує приклад реалізації Hit Test.
- <https://developer.mozilla.org/en-US/docs/Web/API/XRReferenceSpace> – це стаття, яка пояснює що таке Reference Space у WebXR і як воно усе працює.

15. Тестування 3 та 4 завдань можливо лише з використанням вашого мобільного пристрою ([ось інструкція як налаштувати *debugging* для вашого мобільного пристрою](#)). Реалізувавши коректно 3-4 завдання ви матиме схожий результат (рис. 22-23):

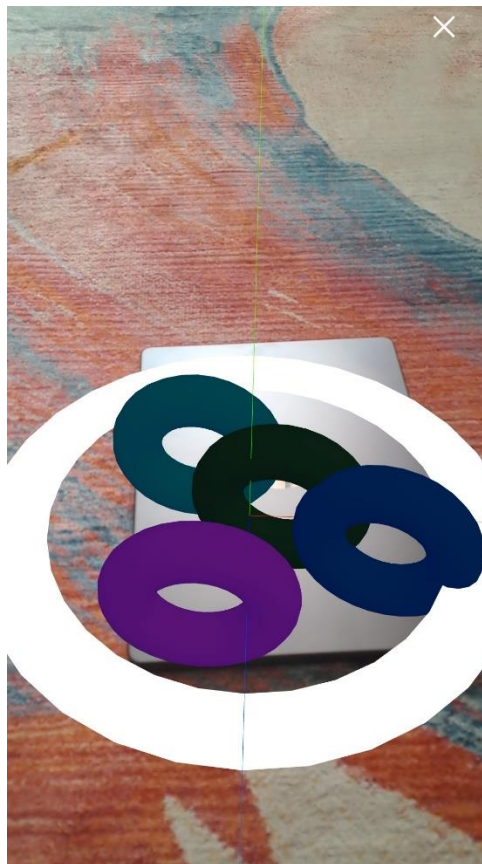


Рис. 22. Тестування третього завдання за допомогою мобільного пристрою

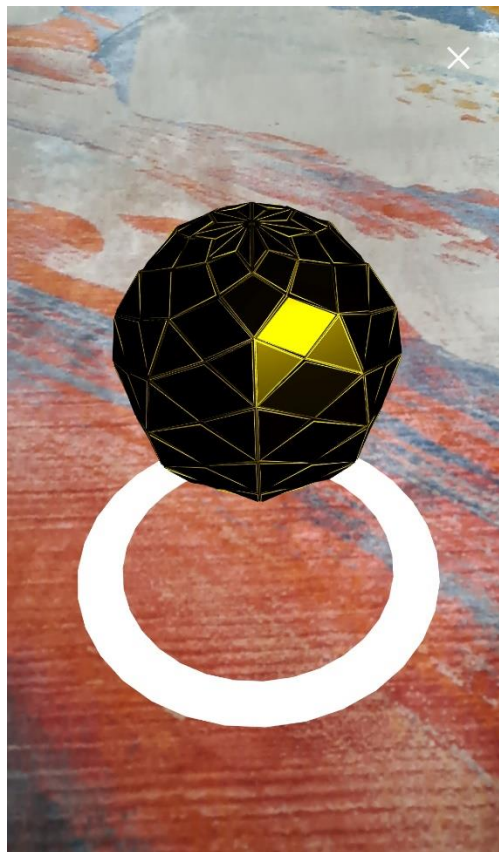


Рис. 23. Тестування четвертого завдання за допомогою мобільного пристрою

- 16.** На цьому етапі лабораторна робота №2 є завершеною. У ході виконання даної лабораторної роботи ви мали змогу ознайомитись з основами розробки доповненої реальності за допомогою WebXR Device API та бібліотеки Three.js. Лабораторний проєкт описує принцип створення базового досвіду доповненої реальності за допомогою WebXR та Three.js; можливості для створення власних 3D-об'єктів та їх розміщення у реальному просторі; можливості для завантаження та використання власних GLTF-моделей з можливістю їх розміщення у реальному просторі та підхід до реалізації технології Hit Test у власних AR-застосунках. Оформіть звіт про виконання поточної лабораторної роботи. **Ваш звіт повинен містити опис виконання кожного завдання та результати тестування ваших AR-застосунків. У звіті має бути лише програмний код реалізації ваших 3D-об'єктів (матеріали, розміри, форми, кольори, анімація).**

Методичні вказівки до лабораторної роботи №2 «Створення досвіду доповненої реальності за допомогою можливостей WebXR Device API та бібліотеки Three.js» з дисципліни «Системи віртуальної та доповненої реальності» для студентів спеціальності 122 «Комп'ютерні науки», освітньої програми «Комп'ютерні науки (Обчислювальний інтелект смарт-систем)» Укл. Глод С. І., Львів: Національний університет «Львівська Політехніка», 2025.

Укладач:

Глод С. І.